

CT520 – Algoritmos e Estruturas de Dados

Curso EN-AEL – 5º Ano

2º Semestre

Docente: Prof. Cortez e Correia

cortez.correia@marinha.pt

Algoritmos lineares de procura.

Objetivos de aprendizagem

- Fornecer conhecimentos para compreender e implementar:
 - algoritmos lineares de procura de dados.

Sumário

- Algoritmos de Procura (*Search Algorithms*):
 1. Procura Sequencial / Linear Search
 2. Procura de Tabela de Dispersão / Hash-Table Search
 3. Procura Binária / Binary Search
 4. Procura Ternária / Ternary Search
 5. Procura por Saltos / Jump Search
 6. Procura Exponencial / Exponential Search
 7. Procura de Interpolação / Interpolation Search
 8. Procura de Fibonacci / Fibonacci Search

Algoritmos de Procura (*Search Algorithms*)

Complexidade Temporal / Espacial

1) Linear Search $O(n)$ $O(1)$
2) Hash Table Search $O(n)$ $O(1)$

3) Binary Search $O(\log_2 n)$ $O(1)$
4) Ternary Search $O(\log_3 n)$ $O(1)$
5) Jump Search $O(\sqrt{n})$ $O(1)$
6) Exponential Search $O(\log_2 n)$ $O(1)$
7) Interpolation Search $O(n)$ $O(1)$
8) Fibonacci Search $O(\log n)$ $O(1)$

0	1	2	3	4	5	6	7	8	9	10	11
27	1	120	10	3	90	25	14	1	34	90	78

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Procurar por:

Valor distinto: 3

Valor duplicado: 90

Valor inexistente: 15

Procura Sequencial (*Linear Search*)

Procurar por:
Valor distinto: 3

LinearSearch(List, 3, 0, 11)

	0	1	2	3	4	5	6	7	8	9	10	11
List	27	1	120	10	3	90	25	14	1	34	90	78

i	0
List[i]	27
3 == 27?	No

$i = i + 1$
→

i	1
List[i]	1
3 == 1?	No

$i = i + 1$
→

i	2
List[i]	120
3 == 120?	No

$i = i + 1$
→

i	3
List[i]	10
3 == 10?	No

$i = i + 1$
→

i	4
List[i]	3
3 == 3?	Yes Stop



Procura Sequencial (*Linear Search*)

Procurar por:
Valor duplicado: 90

LinearSearch(List, 90, 0, 11)

0	1	2	3	4	5	6	7	8	9	10	11
27	1	120	10	3	90	25	14	1	34	90	78

i	0
List[i]	27
90 == 27?	No

$i = i + 1$
→

i	1
List[i]	1
90 == 1?	No

$i = i + 1$
→

i	2
List[i]	120
90 == 120?	No

$i = i + 1$
→

i	3
List[i]	10
90 == 10?	No

$i = i + 1$
→

i	4
List[i]	3
90 == 3?	No

$i = i + 1$
→

i	5
List[i]	90
90 == 90?	Yes Stop

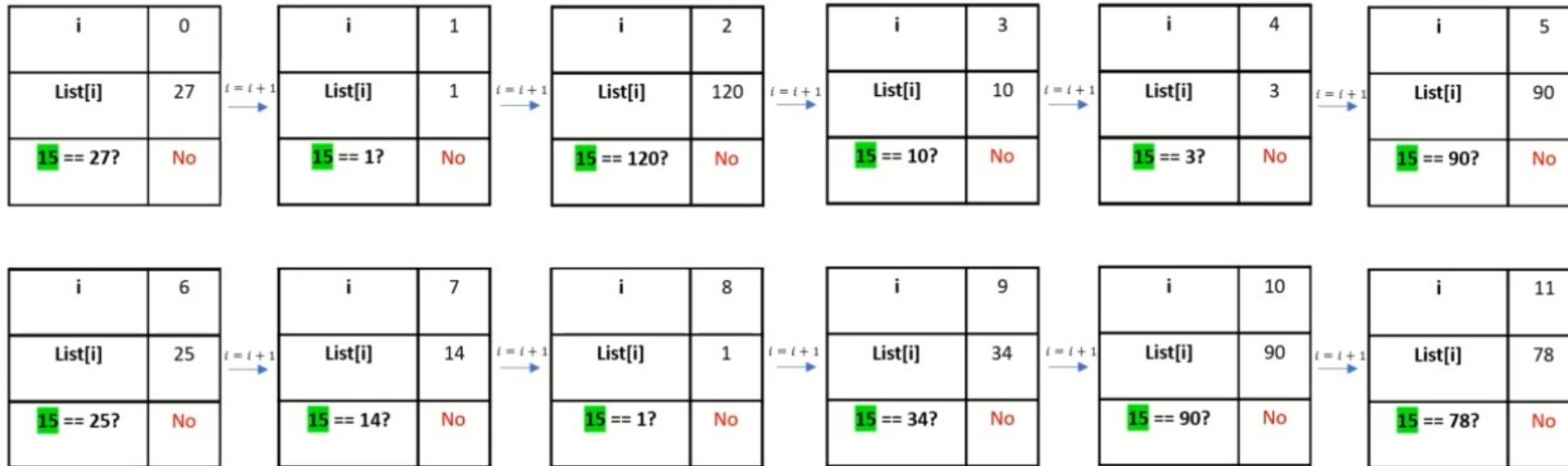


Procura Sequencial (*Linear Search*)

Procurar por:
Valor inexistente: 15

LinearSearch(List, 15, 0, 11)

0	1	2	3	4	5	6	7	8	9	10	11
27	1	120	10	3	90	25	14	1	34	90	78



Procurados todos os elementos da lista,
não existindo o elemento procurado: 15,
retorna -1

Procura Sequencial (*Linear Search*)

`int keyIndex = LinearSearch(List , Key , StartIndex , EndIndex)`

- A forma mais simples de Procura: abordagem *naïve*
- Frequentemente o primeiro algoritmo que vem à mente quando não existe informação adicional sobre a lista.
- **Estratégia:** começar pelo índice mais à esquerda da lista (índice 0) e continuar verificando todos os índices, um a um e linearmente, até encontrar a chave, ou concluir que ela está ausente na lista.
- **Q1. Que índice da lista escolher para iniciar a Procura?**
 - R1. Começar a Procura no primeiro índice da lista (índice 0).
- **Q2. Se o valor no índice for o procurado, parar e retornar o índice. Senão, que índice escolher para examinar a seguir?**
 - R2. Incrementar o índice em 1 e, se o índice exceder o tamanho da lista, parar e retornar -1.
 - ❑ Repetir R2 até que a chave seja encontrada ou tenhamos concluído que não existe na lista.
 - ❑ Na Procura Sequencial, quando a chave está ausente da lista, devemos verificar todos os elementos um a um para poder chegar a essa conclusão. Naturalmente, em listas muito grandes, o processo é ineficiente.

Procura Sequencial: Complexidade

- **Temporal**

- A complexidade temporal da Procura Sequencial é $O(n)$, onde n é o número de elementos na lista. Isso porque, no pior caso, o algoritmo precisará de verificar cada elemento da lista uma vez.

- **Espacial**

- A complexidade espacial da Procura Sequencial é $O(1)$. Isso significa que o espaço necessário para executar o algoritmo é constante e não depende do tamanho da lista de entrada. O algoritmo precisa de um espaço fixo para armazenar variáveis temporárias independentemente do número de elementos na lista que está a ser procurada.

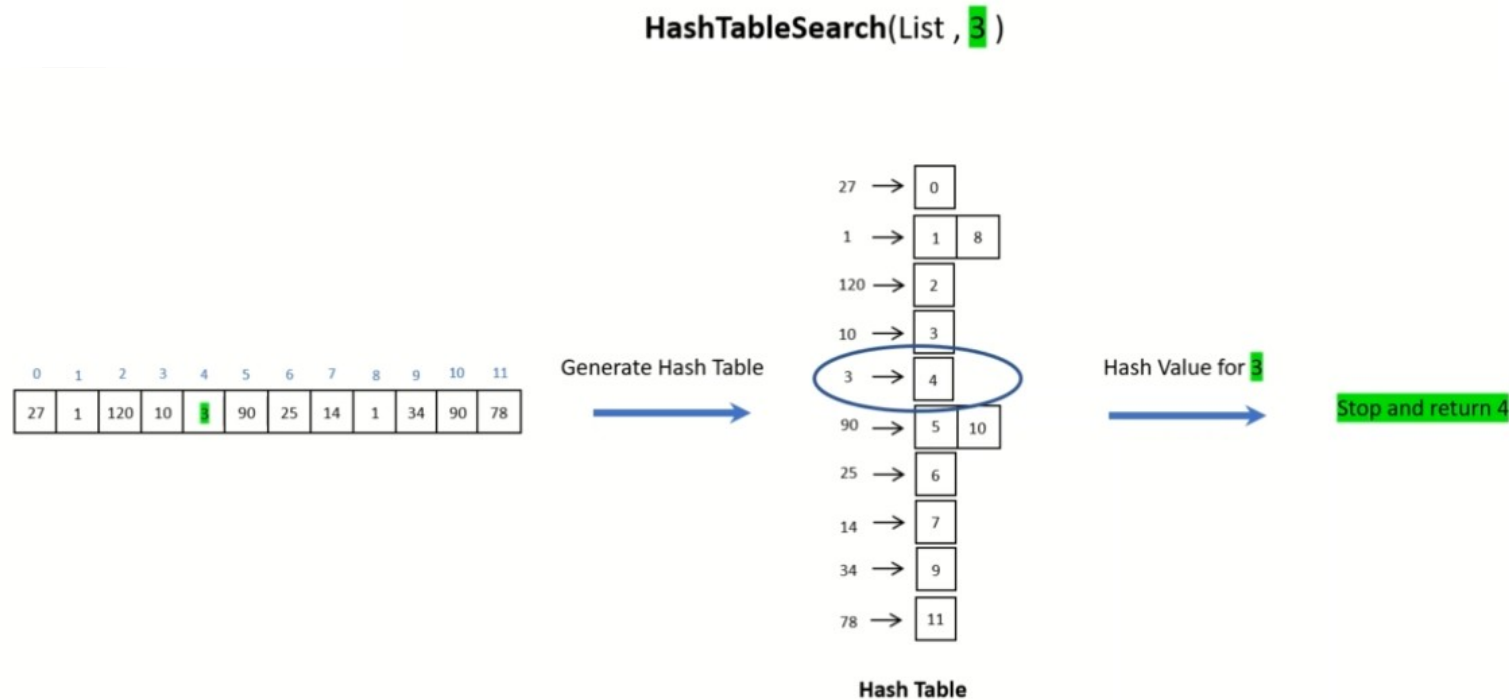
Procura Sequencial (*Linear Search*)

- Implementação em C#

```
public static int LinearSearch(string[] list, string key, int startIndex, int endIndex) {  
    for (int i = startIndex; i <= endIndex; i++) {  
        if (list[i].CompareTo(key) == 0) {  
            return i;  
        }  
    }  
    return -1;  
}
```

Procura em Tabela *Hash* (*HashTable Search*)

Procurar por:
Valor distinto: 3



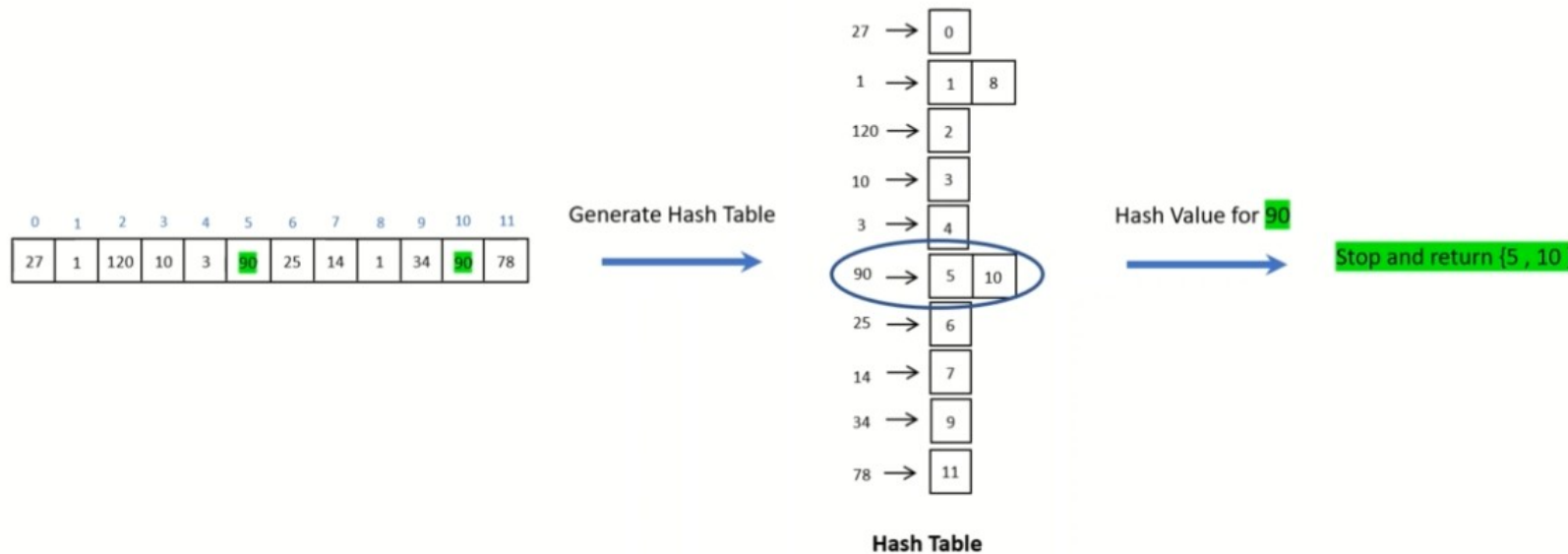
$f(\text{valor})$ – função de *hash* que determina a posição da tabela de *hash* onde se encontram os índices (da tabela original) associados a cada valor



Procura em Tabela *Hash* (*HashTable Search*)

Procurar por:
Valor duplicado: 90

HashTableSearch(List, 90)



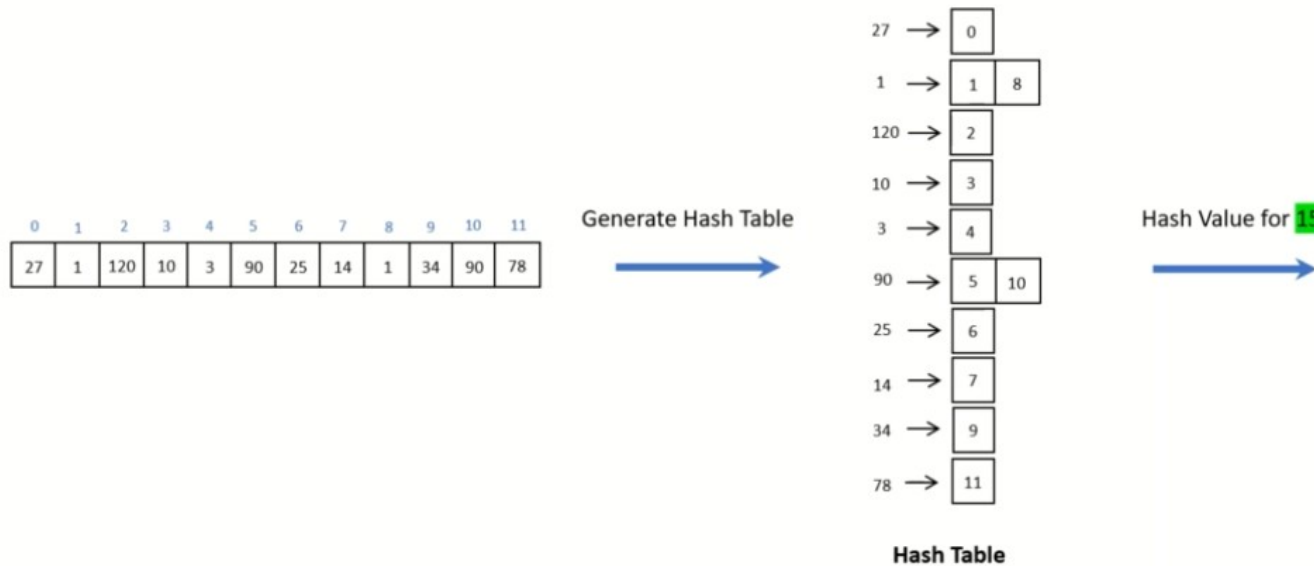
$f(\text{valor})$ – função de *hash* que determina a posição da tabela de *hash* onde se encontram os índices (da tabela original) associados a cada valor



Procura em Tabela *Hash* (*HashTableSearch*)

Procurar por:
Valor inexistente: 15

`HashTableSearch(List, 15)`



A posição correspondente a $f(\text{valor})$ na tabela de hash não contém nenhum índice, logo o valor não existe na tabela original. Não existindo o elemento procurado, retorna -1



$f(\text{valor})$ – função de *hash* que determina a posição da tabela de *hash* onde se encontram os índices (da tabela original) associados a cada valor

Procura em Tabela *Hash* (*HashTableSearch*)

`int keyIndex = HashTableSearch(List ,key)`

- Requer uma etapa de pré-processamento para construir primeiro uma tabela *hash* relativa à tabela inicial.
- A ideia é usar uma função *hash* adequada (colisões mínimas) para calcular os *hashs* dos elementos da tabela inicial e armazenar na tabela *hash* os índices dos valores iniciais.
 - as chaves da tabela de *hash* são valores *hashs* dos elementos na lista, e os valores na tabela de *hash* são índices de elementos na lista original
- Q1. Como iniciar a Procura?
 - R1.
 - Calcular o valor do *hash* para a chave.
 - Se o valor de *hash* calculado estiver ausente da tabela de *hash*, parar e retornar -1.
 - Senão ler o valor (ou lista) de índices correspondente a essa chave *hash*.
 - Iniciar a Procura no primeiro índice dessa lista de índices na tabela principal.
- Q2. Que índice escolher?
 - R2. Se o valor no índice for único, parar e retornar o índice. Caso contrário, examinar o índice seguinte da lista de índices.
 - Repetir R2 até encontrar o valor, retornando então o índice correspondente ao valor.

Procura em Tabela *Hash* (*HashTable Search*)

```
public static List<int> HashTableSearch(string[] list, string key) {  
    Dictionary<int, List<int>> hashTable = ConvertArray2HashTable(list);  
    int keyHash = key.GetHashCode();  
    if (hashTable.ContainsKey(keyHash)) {  
        return hashTable[keyHash];  
    }  
    return new List<int> { -1};  
}
```

```
private static Dictionary<int, List<int>> ConvertArray2HashTable(string[] list) {  
    var hashTable = new Dictionary<int, List<int>>();  
    for (int i = 0; i < list.Length; i++) {  
        int hash = list[i].GetHashCode();  
        if (!hashTable.ContainsKey(hash)) {  
            hashTable.Add(hash, new List<int> { i });  
        }  
        else {  
            hashTable[hash].Add(i);  
        }  
    }  
    return hashTable;  
}
```

Procura em Tabela *Hash*: Complexidade

- **Temporal**

- A complexidade temporal média é $O(1)$, o que significa que, em média, o tempo necessário para localizar um elemento é constante, independentemente do tamanho da tabela. No entanto, no pior caso, se houver muitas colisões e todos os elementos forem mapeados para a mesma posição da tabela, a complexidade pode degradar para $O(n)$.

- **Espacial**

- A complexidade espacial é $O(n)$, onde n é o número de chaves que podem ser armazenadas na tabela. Isso se deve ao fato de que a tabela precisa ter espaço para todas as chaves e possíveis valores, além de estruturas para lidar com colisões, como listas encadeadas no caso de colisões.

Procura Binária (*Binary Search*)

Procurar por:
Valor distinto: 3

$$\text{Middle Index} = \left\lfloor \frac{\text{Start Index} + \text{End Index}}{2} \right\rfloor$$

1 BinarySearch(List, 3, 0, 11)

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Start Index	0
End Index	11
Middle Index	5
Middle Value	25
3 < 25 Next search in the left half	BinarySearch(List, 3, 0, 4)



2 BinarySearch(List, 3, 0, 4)

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Start Index	0
End Index	4
Middle Index	2
Middle Value	3
3 = 3 Search is complete. Stop Index : Middle Index = 2	



Procura Binária (*Binary Search*)

Procurar por:
Valor duplicado: 90

1 BinarySearch(List, 90, 0, 11)

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Start Index	0
End Index	11
Middle Index	5 $(5 = \lfloor \frac{0 + 11}{2} \rfloor)$
Middle Value	25
90 > 25 Next search in the right half	BinarySearch(List, 90, 6, 11)

If right half :
 $\text{BinarySearch}(\text{List}, 90, \text{middleIndex} + 1, \text{endIndex})$



2 BinarySearch(List, 90, 6, 11)

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Start Index	6
End Index	11
Middle Index	8 $(8 = \lfloor \frac{6 + 11}{2} \rfloor)$
Middle Value	78
90 > 78 Next search in the right half	BinarySearch(List, 90, 9, 11)

If right half :
 $\text{BinarySearch}(\text{List}, 90, \text{middleIndex} + 1, \text{endIndex})$

3 BinarySearch(List, 90, 9, 11)

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Start Index	9
End Index	11
Middle Index	10 $(10 = \lfloor \frac{9 + 11}{2} \rfloor)$
Middle Value	90
90 = 90	

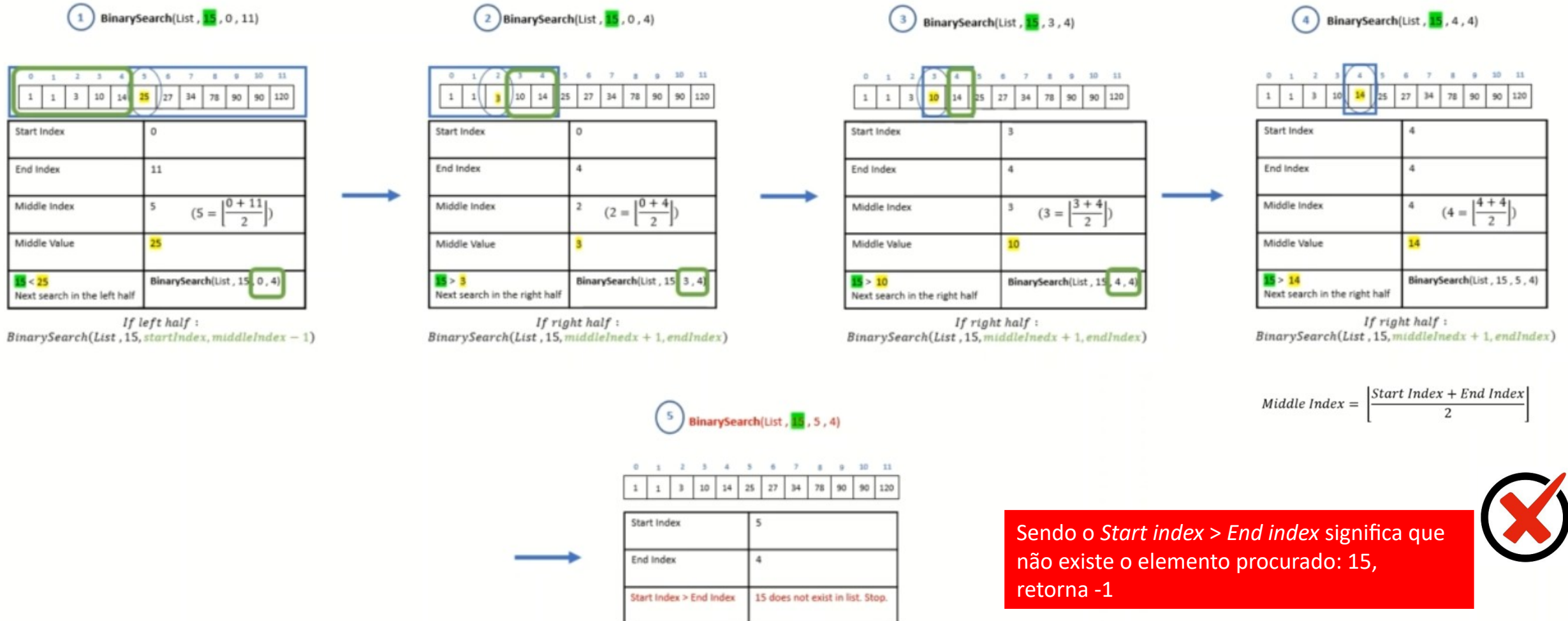


$$\text{Middle Index} = \left\lfloor \frac{\text{Start Index} + \text{End Index}}{2} \right\rfloor$$



Procura Binária (*Binary Search*)

Procurar por:
Valor inexistente: 15



Procura Binária (*Binary Search*)

int keyIndex = BinarySearch(List, Key, StartIndex, EndIndex)

- Dividir a **lista ordenada** em duas sub-listas de tamanhos iguais (daí a designação de procura binária) e com base no resultado da comparação do valor do meio da lista com a *chave* (valor a encontrar), continuar a Procura na sub-lista (da esquerda ou da direita) que poderá conter o elemento.
- Q1. Em que índice da lista iniciamos a Procura?
 - R1: Começar a Procura no meio da lista.
- Q2. Se o valor neste índice corresponder ao procurado, parar e retornar o índice. Caso contrário, que índice examinar a seguir?
 - R2: Se a chave for menor que o elemento do meio, repetir R1 e R2 na metade esquerda da lista atual (se a metade esquerda não contiver o elemento procurado, parar e retornar -1). Caso contrário, se a chave for maior que o elemento do meio, repetir R1 e R2 sobre a metade direita da lista atual (se a metade direita não contiver nenhum elemento, parar e retornar -1).

Procura Binária (*Binary Search*)

```
public static int BinarySearch(string[] list, string key, int startIndex, int endIndex) {  
    if (startIndex > endIndex) {  
        return -1;  
    }  
  
    /* If key is NOT in the range, terminate search. */  
    if (key.CompareTo(list[startIndex]) < 0 || key.CompareTo(list[endIndex]) > 0) {  
        return -1;  
    }  
  
    int middleIndex = (startIndex + endIndex) / 2;  
    string middleValue = list[middleIndex];  
  
    if (key.CompareTo(middleValue) == 0) {  
        return middleIndex;  
    }  
    if (key.CompareTo(middleValue) < 0) {  
        return BinarySearch(list, key, startIndex, middleIndex - 1);  
    }  
    if (key.CompareTo(middleValue) > 0) {  
        return BinarySearch(list, key, middleIndex + 1, endIndex);  
    }  
  
    return -1;  
}
```

Procura Binária: Complexidade

- **Temporal**

- A complexidade temporal é $O(\log n)$, onde n é o número de elementos na lista. Isso acontece porque cada passo da procura binária divide o conjunto de pesquisa a metade, o que significa que o número de elementos a serem verificados é reduzido exponencialmente a cada passo.

- **Espacial**

- A complexidade espacial é $O(1)$, quando implementada de forma iterativa, pois a quantidade de memória utilizada não depende do tamanho da lista de entrada e requer apenas um espaço constante para armazenar os índices ou ponteiros que delimitam a sublista que está sendo pesquisada.

$$\frac{1}{3} \text{Index} = \text{StartIndex} + \left\lfloor \frac{1}{3} \times (\text{EndIndex} - \text{StartIndex}) \right\rfloor$$

$$\frac{2}{3} \text{Index} = \text{StartIndex} + \left\lfloor \frac{2}{3} \times (\text{EndIndex} - \text{StartIndex}) \right\rfloor$$

Procura Ternária (*Ternary Search*)

Procurar por:
Valor distinto: 3

1 TernarySearch(List, 3, 0, 11)

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Start Index	0
End Index	11
1/3 Index	3 $3 = 0 + \left\lfloor \frac{1}{3} \times (11 - 0) \right\rfloor$
1/3 Value	10
2/3 Index	7 $7 = 0 + \left\lfloor \frac{2}{3} \times (11 - 0) \right\rfloor$
2/3 Value	34
3 < (1/3 Value : 10) Next search in the first part	TernarySearch(List, 3, 0, 2)



2 TernarySearch(List, 3, 0, 2)

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Start Index	0
End Index	2
1/3 Index	0 $0 = 0 + \left\lfloor \frac{1}{3} \times (2 - 0) \right\rfloor$
1/3 Value	1
2/3 Index	1 $1 = 0 + \left\lfloor \frac{2}{3} \times (2 - 0) \right\rfloor$
2/3 Value	1
3 > (2/3 Value : 1)	TernarySearch(List, 3, 2, 2)



2 TernarySearch(List, 3, 2, 2)

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Start Index	2
End Index	2
1/3 Index	2 $2 = 2 + \left\lfloor \frac{1}{3} \times (2 - 2) \right\rfloor$
1/3 Value	3
2/3 Index	2 $2 = 2 + \left\lfloor \frac{2}{3} \times (2 - 2) \right\rfloor$
2/3 Value	3
3 = (1/3 & 2/3 Value : 3)	Stop. Search Complete.

If $3 < \frac{1}{3} \text{Value}$: TernarySearch(List, 3, StartIndex, $\frac{1}{3} \text{Index} - 1$)

If $3 > \frac{2}{3} \text{Value}$: TernarySearch(List, 3, $\frac{2}{3} \text{Index} + 1$, EndIndex)



$$\frac{1}{3}Index = StartIndex + \left\lfloor \frac{1}{3} \times (EndIndex - StartIndex) \right\rfloor$$

$$\frac{2}{3}Index = StartIndex + \left\lfloor \frac{2}{3} \times (EndIndex - StartIndex) \right\rfloor$$

Procura Ternária (*Ternary Search*)

Procurar por:
 Valor duplicado: 90

1
 TernarySearch(List , 90 , 0 , 11)

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Start Index	0
End Index	11
1/3 Index	3 $3 = 0 + \left\lfloor \frac{1}{3} \times (11 - 0) \right\rfloor$
1/3 Value	10
2/3 Index	6 $6 = 0 + \left\lfloor \frac{2}{3} \times (11 - 0) \right\rfloor$
2/3 Value	27
90 > (2/3 Value : 27) Next search in the third part	TernarySearch(List , 90 , 7 , 11)

2
 TernarySearch(List , 90 , 7 , 11)

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Start Index	7
End Index	11
1/3 Index	8 $8 = 7 + \left\lfloor \frac{1}{3} \times (11 - 7) \right\rfloor$
1/3 Value	78
2/3 Index	9 $9 = 7 + \left\lfloor \frac{2}{3} \times (11 - 7) \right\rfloor$
2/3 Value	90
90 = (2/3 Value : 90)	Search is complete. Stop. Index : 2/3 Index = 9



If $90 > \frac{2}{3}Value$: TernarySearch(List, 90, $\frac{2}{3}Index + 1$, EndIndex)

$$\frac{2}{3}Index = StartIndex + \left\lfloor \frac{2}{3} \times (EndIndex - StartIndex) \right\rfloor$$

Procurar por:
Valor inexistente: 15



Sendo o *Start index* > *End index* significa que não existe o elemento procurado: 15, retorna -1



Procura Ternária (*Ternary Search*)

`int keyIndex = TernarySearch(List, Key, StartIndex, EndIndex)`

- Dividir a lista ordenada em três sub-listas de tamanhos aproximadamente iguais (daí a designação da procura) e, com base nos valores em $n/3$ e $2n/3$ da lista e sua comparação com a chave (valor a procurar), continuar a Procura na sub-lista que provavelmente conterá a chave.
- Q1. Que índice da lista escolher para iniciar a Procura?
 - R1: Calcular dois índices pelos quais a lista é dividida em três sub-listas de tamanhos aproximadamente iguais. Começar a Procura nesses dois índices ($n/3$ e $2n/3$).
- Q2. Se o valor em qualquer dos índices corresponder, parar e retornar o índice. Caso contrário, que índice escolher examinar a seguir?
 - R2: Se a chave for menor do que o valor em $n/3$, repetir as etapas R1 e R2 no primeiro terço da lista. Caso contrário, se a chave estiver entre os valores de $n/3$ a $2n/3$, repetir R1 e R2 no segundo terço da lista. Caso contrário, repetir R1 e R2 no terceiro terço da lista.

Procura Ternária (*Ternary Search*)

```
public static int TernarySearch(string[] list, string key, int startIndex, int endIndex) {
    if (startIndex > endIndex) {
        return -1;
    }

    /* If key is NOT in the range, terminate search. */
    if (key.CompareTo(list[startIndex]) < 0 || key.CompareTo(list[endIndex]) > 0) {
        return -1;
    }

    /* Dividing list by ((endIndex - startIndex) / 3) size in to 3 sections. */
    int oneThirdIndex = startIndex + (endIndex - startIndex) * 1 / 3;
    int twoThirdIndex = startIndex + (endIndex - startIndex) * 2 / 3;

    string oneThirdValue = list[oneThirdIndex];
    string twoThirdValue = list[twoThirdIndex];

    if (key.CompareTo(oneThirdValue) == 0) {
        return oneThirdIndex;
    }

    if (key.CompareTo(twoThirdValue) == 0) {
        return twoThirdIndex;
    }

    if (key.CompareTo(oneThirdValue) < 0) {
        return TernarySearch(list, key, startIndex, oneThirdIndex - 1);
    }

    if (key.CompareTo(oneThirdValue) > 0 && key.CompareTo(twoThirdValue) < 0) {
        return TernarySearch(list, key, oneThirdIndex + 1, twoThirdIndex - 1);
    }

    if (key.CompareTo(twoThirdValue) > 0) {
        return TernarySearch(list, key, twoThirdIndex + 1, endIndex);
    }

    return -1;
}
```

Procura Ternária: Complexidade

- **Temporal**

- A complexidade temporal é $O(\log n)$, onde n é o número de elementos na lista ou o intervalo de busca. A procura ternária geralmente realiza mais comparações do que a procura binária em cada iteração, o que pode torná-la ligeiramente menos eficiente.

- **Espacial**

- A complexidade espacial é $O(1)$, pois o algoritmo usa uma quantidade fixa de memória, independentemente do tamanho da lista.

$$\text{Jump Step Length} = \lfloor \sqrt{n} \rfloor$$

Procura em Salto (*Jump Search*)

Procurar por:
Valor distinto: 3

JumpSearch(List, 3, 0, 11)
Jump Step Length = $\lfloor \sqrt{12} \rfloor = 3$

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

i	0
1 < 3	Next: Jump

$$i = i + \lfloor \sqrt{12} \rfloor$$



0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

i	3
10 > 3 Next look backward	LinearSearch (List, 3, $i - \sqrt{12} + 1$, i-1) → LinearSearch (List, 3, 1, 2)

LinearSearch(List, 3, 1, 2)

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

i	1
List[i]	1
3 == 1?	No



i	2
List[i]	3
3 == 3?	Yes Stop



$$\text{Jump Step Length} = \lfloor \sqrt{n} \rfloor$$

Procura em Salto (*Jump Search*)

Procurar por:
Valor duplicado: 90

JumpSearch(List, 90, 0, 11)
Jump Step Length = $\lfloor \sqrt{12} \rfloor = 3$

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

i	0
1 < 90	Next: Jump

$$i = i + \sqrt{12}$$

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

i	3
10 < 90	Next: Jump

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

i	6
27 < 90	Next: Jump

$$i = i + \sqrt{12}$$

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

i	9
90 = 90	Stop



$$\text{Jump Step Length} = \lfloor \sqrt{n} \rfloor$$

Procura em Salto (*Jump Search*)

Procurar por:
Valor inexistente: 15

JumpSearch(List, 15, 0, 11)
Jump Step Length = $\lfloor \sqrt{12} \rfloor = 3$

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

i	0
1 < 15	Next: Jump

$$i = i + \sqrt{12}$$

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

i	3
10 < 15	Next: Jump

Procurados todos os elementos da lista, não existindo o elemento procurado: 15, retorna -1

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

i	6
27 > 15 Next look backward	LinearSearch(List, 15, i- $\sqrt{12}$ + 1, i-1) → LinearSearch(List, 15, 4, 5)

$$i = i + \sqrt{12}$$

LinearSearch(List, 15, 4, 5)

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

i	4
List[i]	14
15 == 14?	No

i	5
List[i]	25
15 == 25?	No



Procura em Salto (*Jump Search*)

`int keyIndex = JumpSearch(List, Key, StartIndex, EndIndex)`

- A ideia é restringir uma lista ordenada numa sub-lista (*container*) que deverá conter a chave (valor a encontrar) e, em seguida, executar uma Procura Sequencial sobre essa sub-lista.
- Para encontrar a sub-lista *container*, o algoritmo salta (daí a designação da procura) por cima dos valores menores que a chave por um passo constante, até encontrar um elemento que é igual ou maior que a chave. Se for igual a Procura é interrompida, senão, dado que a lista é ordenada, se a chave existir na lista, ela deve aparecer antes do elemento que é maior que a chave. Portanto, a sub-lista *container* termina no índice deste elemento e começa no índice seguinte ao que foi verificado na etapa anterior.
- Q1. Em qual índice da lista iniciar a Procura?
 - R1. Começar a Procura no primeiro índice da lista.
- Q2. Se o valor nesse índice coincidir com a chave, parar e retornar o índice. Caso contrário, que índice selecionar para examinar a seguir?
 - R2. Se a chave for maior que o elemento neste índice, incrementar o índice com $\sqrt{n}$, onde n é o tamanho da lista. Caso contrário, se a chave for menor que o elemento nesse índice, executar uma Procura Sequencial na sub-lista *container*:

$$[index - \sqrt{n} + 1 \quad index - 1]$$

Procura em Salto (*Jump Search*)

```
public static int JumpSearch(string[] list, string key) {  
    /* If key is NOT in the range, terminate search. */  
    if (key.CompareTo(list[0]) < 0 || key.CompareTo(list[list.Length - 1]) > 0) {  
        return -1;  
    }  
  
    int jumpStepLength = (int)Math.Floor(Math.Sqrt(list.Length));  
  
    int nextIndex = 0;  
    while (list[nextIndex].CompareTo(key) < 0) {  
        nextIndex += jumpStepLength; // Jump forward  
        if (nextIndex >= list.Length) {  
            nextIndex = list.Length - 1;  
            break;  
        }  
    }  
  
    if (list[nextIndex].CompareTo(key) == 0) {  
        return nextIndex;  
    }  
  
    int linearSearchStartIndex = nextIndex - jumpStepLength + 1; // Jump backward.  
    if (linearSearchStartIndex < 0) {  
        return -1;  
    }  
    return LinearSearch(list, key, linearSearchStartIndex, nextIndex - 1);  
}
```

Procura em Salto: Complexidade

- **Temporal**

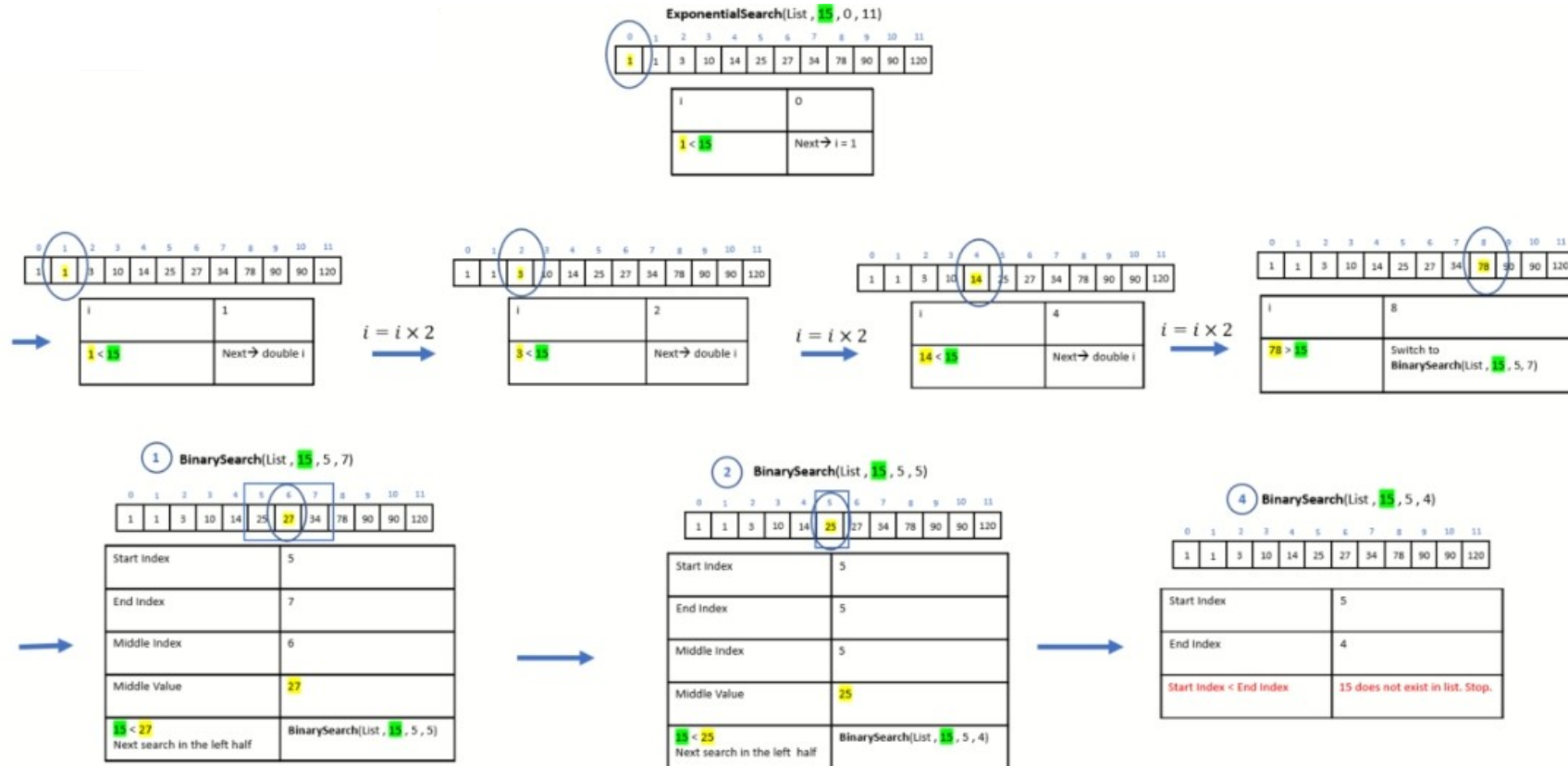
- A complexidade temporal média é $O(\sqrt{n})$, onde n é o número de elementos na lista. Isso ocorre porque, ao percorrer a lista em saltos de tamanho m , são feitos no máximo n/m comparações, e, uma vez que um intervalo contendo o valor é encontrado, são feitas no máximo m comparações adicionais numa procura Sequencial. Embora não seja tão eficiente quanto a procura binária em termos de número de comparações, a procura em salto pode ser mais rápida em casos onde o tempo de acesso à memória é um fator limitante (por exemplo, em discos rígidos ou outros meios de armazenamento não volátil), pois tende a causar menos falhas de cache devido à sua abordagem sequencial de "saltos".

- **Espacial**

- A complexidade espacial é $O(1)$, uma vez que o algoritmo precisa apenas de uma quantidade constante de espaço para armazenar os índices e realizar as comparações.

Procura exponencial (*Exponential Search*)

Procurar por:
Valor inexistente: 15



Procurados todos os elementos da lista, não existindo o elemento procurado: 15, retorna -1



Procura exponencial (*Exponential Search*)

```
public static int ExponentialSearch(string[] list, string key) {  
    /* If key is NOT in the range, terminate search. */  
    if (key.CompareTo(list[0]) < 0 || key.CompareTo(list[list.Length - 1]) > 0) {  
        return -1;  
    }  
  
    if (list[0].CompareTo(key) == 0) {  
        return 0;  
    }  
  
    /* Ideally should start from index 0, however that would make the while loop indexing complex,  
    * thus treating index 0 differently, and then continuing with the rest. */  
    int nextIndex = 1;  
    /* multiple the search step by 2, until encountering an element that is bigger than the key. */  
    while (nextIndex < list.Length && list[nextIndex].CompareTo(key) < 0) {  
        nextIndex *= 2;  
    }  
  
    if (nextIndex >= list.Length) {  
        nextIndex = list.Length - 1;  
    }  
    if (list[nextIndex].CompareTo(key) == 0) {  
        return nextIndex;  
    }  
  
    /* The range at which the key is expected to be is thus [(nextIndex/2)+1, nextIndex-1]  
    * perform a binary search over this range. */  
    return BinarySearch(list, key, nextIndex / 2 + 1, nextIndex - 1);  
}
```

Procura exponencial: Complexidade

- **Temporal**

- A complexidade temporal é **$O(\log n)$** , onde n é a posição do elemento-alvo. Isso acontece porque a busca exponencial primeiro encontra o intervalo de tamanho 2^i que contém o elemento-alvo em $O(\log n)$ tempo (onde i é o menor índice tal que o elemento na posição 2^i é maior ou igual ao elemento-alvo) e, em seguida, executa uma busca binária dentro deste intervalo, que também é $O(\log n)$. É um método eficiente para localizar elementos em listas grandes e ordenadas, oferecendo uma boa performance especialmente quando o elemento-alvo está mais próximo do início da lista.

- **Espacial**

- A complexidade espacial é **$O(1)$** , porque o algoritmo usa uma quantidade constante de espaço, independentemente do tamanho da lista que está sendo pesquisada.

$$startIndex + \left\lfloor (endIndex - startIndex) \times \frac{key - List[startIndex]}{List[endIndex] - List[startIndex]} \right\rfloor$$

Procura por interpolação (*Interpolation Search*)

Procurar por:
Valor distinto: 3



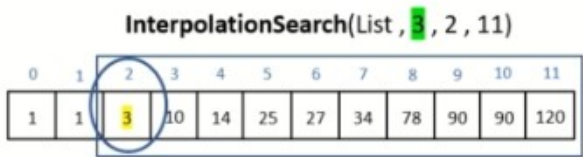
Start Index	0
End Index	11
Index	$0 = 0 + \left\lfloor (11 - 0) \times \frac{3 - 1}{120 - 1} \right\rfloor$
Index Value	1
$3 > 1$	InterpolationSearch(List, 3, 1, 11)

Next look to the right of List[0]



Start Index	1
End Index	11
Index	$1 = 1 + \left\lfloor (11 - 1) \times \frac{3 - 1}{120 - 1} \right\rfloor$
Index Value	1
$3 > 1$	InterpolationSearch(List, 3, 2, 11)

Next look to the right of List[1]



Start Index	2
End Index	11
Index	$2 = 2 + \left\lfloor (11 - 2) \times \frac{3 - 3}{120 - 3} \right\rfloor$
Index Value	3
$3 = 3$	



$$startIndex + \left\lfloor (endIndex - startIndex) \times \frac{key - List[startIndex]}{List[endIndex] - List[startIndex]} \right\rfloor$$

Procura por interpolação (*Interpolation Search*)

Procurar por:
Valor duplicado: 90

InterpolationSearch(List, 90, 0, 11)

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Start Index	0
End Index	11
Index	8 $8 = 0 + \left\lfloor (11 - 0) \times \frac{90 - 1}{120 - 1} \right\rfloor$
Index Value	78
$90 > 78$	InterpolationSearch(List, 90, 9, 11)

Next look to the right of List[8]

InterpolationSearch(List, 90, 9, 11)

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Start Index	9
End Index	11
Index	9 $9 = 9 + \left\lfloor (11 - 9) \times \frac{90 - 90}{120 - 90} \right\rfloor$
Index Value	90
$90 = 90$	Search is complete. Stop.



$$startIndex + \left\lfloor (endIndex - startIndex) \times \frac{key - List[startIndex]}{List[endIndex] - List[startIndex]} \right\rfloor$$

Procura por interpolação (*Interpolation Search*)

Procurar por:
Valor inexistente: 15

InterpolationSearch(List, 15, 0, 11)

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Start Index	0
End Index	11
Index	1 $1 = 0 + \left\lfloor (11 - 0) \times \frac{15 - 1}{120 - 1} \right\rfloor$
Index Value	1
$15 > 1$	InterpolationSearch(List, 15, 2, 11)

Next look to the right of List[0]

InterpolationSearch(List, 15, 2, 11)

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Start Index	2
End Index	11
Index	2 $2 = 2 + \left\lfloor (11 - 2) \times \frac{15 - 3}{120 - 3} \right\rfloor$
Index Value	3
$15 > 3$	InterpolationSearch(List, 15, 3, 11)

Next look to the right of List[2]

InterpolationSearch(List, 15, 3, 11)

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Start Index	3
End Index	11
Index	3 $3 = 3 + \left\lfloor (11 - 3) \times \frac{15 - 10}{120 - 10} \right\rfloor$
Index Value	10
$15 > 10$	InterpolationSearch(List, 15, 4, 11)

Next look to the right of List[3]

InterpolationSearch(List, 15, 4, 11)

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Start Index	4
End Index	11
Index	4 $4 = 4 + \left\lfloor (11 - 4) \times \frac{15 - 14}{120 - 14} \right\rfloor$
Index Value	14
$15 > 14$	InterpolationSearch(List, 15, 5, 11)

Next look to the right of List[4]

InterpolationSearch(List, 15, 5, 11)

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Start Index	5
End Index	11
$15 < List[Start Index]$	15 is missing in List. Stop

Procurados os elementos da lista, encontrando um superior ao elemento procurado: 15, retorna -1



Procura por interpolação (*Interpolation Search*)

`int keyIndex = InterpolationSearch(List, key, StartIndex, EndIndex)`

- É efetuada sobre uma lista ordenada e uniformemente distribuída.
- Como o nome sugere, o algoritmo move-se para frente e para trás entre dois polos: o início e o fim da lista.
- Para calcular o índice em que se inicia a Procura, este algoritmo usa o valor da chave relativamente aos dois polos. Se a chave está mais próxima do índice inicial, escolhe um primeiro índice mais próximo desse índice inicial e, da mesma forma, para o índice final.
- Q1. Em que índice da lista se inicia a procura?

- R1: Começar a procura em primeiro lugar em:

$$startIndex + \left\lfloor (endIndex - startIndex) \times \frac{key - List[startIndex]}{List[endIndex] - List[startIndex]} \right\rfloor$$

(onde `startIndex = 0`, `endIndex = n-1`)

- Q2. Se o valor nesse índice for o procurado, parar e retornar o índice. Caso contrário, que índice escolher a seguir?
 - R2: Se a chave for menor que o elemento no índice, repetir R1 e R2 na sub-lista à esquerda do índice (se esta sub-lista não tiver nenhum elemento, parar e retornar -1). Caso contrário, se a chave for maior do que o elemento no índice, repetir R1 e R2 na sub-lista à direita do índice (se esta sublista não tiver nenhum elemento parar e retornar -1).

Procura por interpolação (*Interpolation Search*)

```
public static int InterpolationSearch(int[] list, int key, int startIndex, int endIndex) {
    if (startIndex > endIndex) {
        return -1;
    }

    /* If key is NOT in the range, terminate search.
     * Since the input list is sorted this early check is feasible. */
    if (key.CompareTo(list[startIndex]) < 0 || key.CompareTo(list[endIndex]) > 0) {
        return -1;
    }

    int searchStartIndex = GetStartIndex(list, key, startIndex, endIndex);
    if (!(searchStartIndex >= startIndex && searchStartIndex <= endIndex)) {
        return -1;
    }

    int searchStartValue = list[searchStartIndex];

    if (key.CompareTo(searchStartValue) == 0) {
        return searchStartIndex;
    }

    if (key.CompareTo(searchStartValue) < 0) {
        return InterpolationSearch(list, key, startIndex, searchStartIndex - 1);
    }

    if (key.CompareTo(searchStartValue) > 0) {
        return InterpolationSearch(list, key, searchStartIndex + 1, endIndex);
    }

    return -1;
}
```

```
private static int GetStartIndex(int[] list, int key, int startIndex, int endIndex) {
    double distanceFromStartIndex = ((dynamic)key - (dynamic)list[startIndex]) /
        (double)((dynamic)list[endIndex] - (dynamic)list[startIndex]);
    distanceFromStartIndex *= (endIndex - startIndex);
    int index = (int)(startIndex + distanceFromStartIndex);
    return index;
}
```

Procura por interpolação: Complexidade

- **Temporal**

- A complexidade temporal é $O(n)$, onde n é o número de elementos na lista, no pior caso (por exemplo, quando os elementos não estão uniformemente distribuídos), a complexidade pode degradar para uma performance similar à procura Sequencial. É $O(\log n)$ se os elementos estão uniformemente distribuídos. Isso ocorre porque o algoritmo pode aproximar a posição do elemento-alvo de maneira eficiente.

- **Espacial**

- A complexidade espacial é $O(1)$, o que significa que o espaço necessário para executar o algoritmo é constante e não depende do tamanho da lista de entrada.

$\text{Index} = \text{Min}(\text{EndIndex}, (\text{StartIndex} + [\text{Fib} - 2]))$

Procura de Fibonacci (*Fibonacci Search*)

Procurar por:
Valor distinto: 3

FibonacciSearch(List, 3)
 $n = 12 \rightarrow \text{Fib} = 13$

Fibs: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, ...

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Fib	13
Fib - 1	8
Fib - 2	5
Start Index	-1
Index	4 $4 = \text{Min}(11, (-1 + 5))$
3 < 14	Next : Shift Fib backward twice

Shift Fib backward twice

Fibs: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, ...

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Fib	5
Fib - 1	3
Fib - 2	2
Start Index	-1 (did not change)
Index	1 $1 = \text{Min}(11, (-1 + 2))$
3 > 1	Next : Shift Fib backward once Set Start Index : Index

Shift Fib backward once

Fibs: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, ...

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Fib	3
Fib - 1	2
Fib - 2	1
Start Index	1 (= Index of previous step)
Index	2 $2 = \text{Min}(11, (1 + 1))$
3 = 3	Equal. Stop Search

Start Index = Index
Shift Fib backward once
And move start index forward



$$\text{Index} = \text{Min}(\text{EndIndex}, (\text{StartIndex} + [\text{Fib} - 2]))$$

Procura de Fibonacci (*Fibonacci Search*)

Procurar por:
Valor duplicado: 90

Fibs: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, ...

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Fib	13
Fib - 1	8
Fib - 2	5
Start Index	-1
Index	4 $4 = \text{Min}(11, (-1 + 5))$
90 > 14	Next : Shift Fib backward <u>once</u>

FibonacciSearch(List, 90)
 $n = 12 \rightarrow \text{Fib} = 13$

Start Index = Index
→
Shift Fib backward once
And move start index forward

Fibs: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, ...

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Fib	8
Fib - 1	5
Fib - 2	3
Start Index	4 (= Index of previous step)
Index	7 $7 = \text{Min}(11, (4 + 3))$
90 > 34	Next : Shift Fib backward <u>once</u>

Fibs: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, ...

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Start Index = Index
→
Shift Fib backward once
And move start index forward

Fib	5
Fib - 1	3
Fib - 2	2
Start Index	7 (= Index of previous step)
Index	9 $9 = \text{Min}(11, (7 + 2))$
90 = 90	Equal. Stop Search



$$\text{Index} = \text{Min}(\text{EndIndex}, (\text{StartIndex} + [\text{Fib} - 2]))$$

Procura de Fibonacci (*Fibonacci Search*)

Procurar por:
Valor inexistente: 15

Fibs: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, ...

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Fib	13
Fib - 1	8
Fib - 2	5
Start Index	-1
Index	4 $4 = \text{Min}(11, (-1 + 5))$
15 > 14	Next : Shift Fib backward <u>once</u>

FibonacciSearch(List, 15)
 $n = 12 \rightarrow \text{Fib} = 13$

Start Index = Index
→
Shift Fib backward once
And move start index forward

Fibs: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, ...

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Fib	8
Fib - 1	5
Fib - 2	3
Start Index	4 (= Index of previous step)
Index	7 $7 = \text{Min}(11, (4 + 3))$
15 < 34	Shift Fib backward <u>twice</u>

Fibs: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, ...

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Fib	3
Fib - 1	2
Fib - 2	1
Start Index	4 (did not change)
Index	5 $5 = \text{Min}(11, (4 + 1))$
15 < 25	Shift Fib backward <u>twice</u>

→
Shift Fib backward twice

Fibs: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, ...

0	1	2	3	4	5	6	7	8	9	10	11
1	1	3	10	14	25	27	34	78	90	90	120

Fib	1
Fib - 1	1
Fib - 2	0
Start Index	4 (did not change)
Index	4 $4 = \text{Min}(11, (4 + 0))$
15 > 14 & Fib <= 1	15 is missing in the array.

Procurados todos os elementos da lista, não existindo o elemento procurado: 15, retorna -1



Procura de Fibonacci (*Fibonacci Search*)

int keyIndex = FibonacciSearch(List, key, StartIndex, EndIndex)

- Procura numa **lista ordenada**, utilizando a sequência de *Fibonacci* para realizar a procura.
 - A sequência de *Fibonacci* é a seguinte: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ...
 - A fórmula usada para gerar a sequência de *Fibonacci* é: $F_k = F_{k-1} + F_{k-2}$.
 - Por exemplo, com $k = 2$: $F_2 = F_1 + F_0 = 1 = 1 + 0$
- Para iniciar a procura devemos primeiro encontrar o menor número de *Fibonacci* (F_n) que seja maior ou igual a n (tamanho da lista). Por exemplo:
para $n = 12 \rightarrow F_n = 13$; para $n = 21 \rightarrow F_n = 21$.
- Q1. Em que índice da lista se inicia a procura?

Index = Min(EndIndex, (StartIndex + [Fib — 2]))

 - R1: Começar a procura no índice dado pela formula, tendo em conta o número *Fibonacci* F_{n-2} [$F_n = F_{n-1} + F_{n-2}$]
- Q2. Se o valor nesse índice for o procurado, parar e retornar o índice. Caso contrário, que índice escolher examinar a seguir?
 - R2: Se F_{n-2} for 0, parar e retornar -1. Caso contrário, se a **chave for menor** que o elemento do índice calculado, deslocar F_n para **trás duas vezes (mantendo o valor de StartIndex)**, caso a **chave seja maior**, deslocar F_n para **trás uma vez (atualizando o valor de StartIndex)**. No início StartIndex começa com o valor -1. EndIndex é o índice do último elemento da lista.
 - Repetir R2 até obter um dos resultados finais.

Procura de Fibonacci (*Fibonacci Search*)

```
public static int FibonacciSearch(string[] list, string key) {
    /* If key is NOT in the range, terminate search.
     * Since the input list is sorted this early check is feasible. */
    if (key.CompareTo(list[0]) < 0 || key.CompareTo(list[list.Length - 1]) > 0) {
        return -1;
    }

    FibonacciElement fib = GetSmallestFibonacciBiggerThanNumber(list.Length);
    int startIndex = -1;

    /* Note that fib numbers indicate indexes in the list to look at, and not values. */
    /* meaning in the sequence {0, 1, 1, 2, 3, ...} the while loop will stop when
     * fibN = 2, thus fibN2 can at least be 1 */
    while (fib.FibN > 1)
    {
        /* First compare to the value at index FibN2 */
        int index = Math.Min(startIndex + fib.FibN2, list.Length - 1);
        if (key.CompareTo(list[index]) == 0) {
            return index;
        }

        if (key.CompareTo(list[index]) < 0) {
            // Shift backward twice as checked against FibN2
            fib.ShiftBackward();
            fib.ShiftBackward();
        }

        if (key.CompareTo(list[index]) > 0) {
            fib.ShiftBackward();
            startIndex = index; /* Which is the previous Fib2*/
        }
    }

    if (fib.FibN1 == 1 && (startIndex + 1) < list.Length &&
        list[startIndex + 1].CompareTo(key) == 0) {
        return startIndex + 1;
    }

    return -1;
}
```

```
class FibonacciElement {
    6 references
    public int FibN { get; set; }
    8 references
    public int FibN1 { get; set; }
    6 references
    public int FibN2 { get; set; }
    1 reference
    public FibonacciElement(int fibN2, int fibN1) {
        FibN2 = fibN2;
        FibN1 = fibN1;
        FibN = FibN1 + FibN2;
    }
    // Shifts Fibonacci number one element forward in the sequence.
    1 reference
    public void ShiftForward() {
        FibN2 = FibN1;
        FibN1 = FibN;
        FibN = FibN1 + FibN2;
    }
    // Shifts Fibonacci number one element backward in the sequence.
    0 references
    public void ShiftBackward() {
        FibN = FibN1;
        FibN1 = FibN2;
        FibN2 = FibN - FibN1;
    }
}
```

```
private static FibonacciElement GetSmallestFibonacciBiggerThanNumber(int number) {
    var fib = new FibonacciElement(0, 1);
    while (fib.FibN < number) {
        fib.ShiftForward();
    }
    return fib;
}
```


Procura de Fibonacci: Complexidade

- **Temporal**

- A complexidade temporal média é $O(\log n)$, onde n é o tamanho da lista. Isso ocorre porque em cada etapa do algoritmo, este divide a lista usando os números de Fibonacci, que crescem exponencialmente, reduzindo assim o espaço de busca de maneira eficiente.

- **Espacial**

- A complexidade espacial é $O(1)$, pois como a busca binária, não requer espaço adicional significativo além do armazenamento para um número finito de índices e variáveis de controle.

Anexos

Programa de execução de algoritmos de procura

```
static void Main(string[] args) {  
  
    string[] nomes = new string[] { "Joana", "Pedro", "Antonio", "Zacarias", "Bernardo", "Artur", // 0 - 5  
                                     "Maria", "Zacarias", "Afonso", "João", "Pedro", "Joana", // 6 - 11  
                                     "Pedro", "Antonio", "Zulmira"}; // 12 - 14  
  
    // Linear Search  
    Console.WriteLine(LinearSearch(nomes, "Zacarias", 0, nomes.Length - 1));  
  
    // Linear Search  
    foreach (int i in HashTableSearch(nomes, "Zacarias"))  
        Console.WriteLine(i + " ");  
    Console.WriteLine();  
  
    Array.Sort(nomes);  
    foreach (string s in nomes)  
        Console.WriteLine(s + " ");  
    Console.WriteLine();  
  
    // Binary Search  
    Console.WriteLine(BinarySearch(nomes, "Zacarias", 0, nomes.Length - 1));  
    Console.WriteLine();  
  
    // Ternary Search  
    Console.WriteLine(TernarySearch(nomes, "Zacarias", 0, nomes.Length - 1));  
    Console.WriteLine();  
  
    // Jump Search  
    Console.WriteLine(JumpSearch(nomes, "Zacarias"));  
    Console.WriteLine();  
}
```

```
// Fibonacci Search  
Console.WriteLine(FibonacciSearch(nomes, "Zacarias"));  
Console.WriteLine();  
  
// Exponential Search  
Console.WriteLine(ExponentialSearch(nomes, "Zacarias"));  
Console.WriteLine();  
  
Array.Sort(numeros);  
foreach (int i in numeros)  
    Console.WriteLine(i + " ");  
Console.WriteLine();  
  
// Interpolation Search  
Console.WriteLine(InterpolationSearch(numeros, 77, 0, numeros.Length - 1));  
Console.WriteLine();  
}
```

```
3  
3 7  
Afonso Antonio Antonio Artur Bernardo Joana Joana João Maria Pedro Pedro Pedro Zacarias Zacarias Zulmira  
13  
  
12  
  
12  
  
12  
  
12  
  
0 8 9 14 20 20 33 54 77 90 100  
8
```

Exercício de procura

- Fazer um programa para gerar um conjunto de contas bancárias e depois procurar uma conta bancária específica mostrando os seus dados. Fazer um depósito e um levantamento. Implemente a conta bancária conforme diagrama e interações abaixo:

```
Conta a procurar: Number: 09 Holder: Tilular 09 Balance: 500.00 WithdrawLimit: 100.00
```

```
Contas em carteira:
```

```
Number: 10 Holder: Tilular 10 Balance: 500.00 WithdrawLimit: 100.00
Number: 04 Holder: Tilular 04 Balance: 500.00 WithdrawLimit: 100.00
Number: 05 Holder: Tilular 05 Balance: 500.00 WithdrawLimit: 100.00
Number: 03 Holder: Tilular 03 Balance: 500.00 WithdrawLimit: 100.00
Number: 06 Holder: Tilular 06 Balance: 500.00 WithdrawLimit: 100.00
Number: 08 Holder: Tilular 08 Balance: 500.00 WithdrawLimit: 100.00
Number: 02 Holder: Tilular 02 Balance: 500.00 WithdrawLimit: 100.00
Number: 09 Holder: Tilular 09 Balance: 500.00 WithdrawLimit: 100.00
Number: 07 Holder: Tilular 07 Balance: 500.00 WithdrawLimit: 100.00
Number: 01 Holder: Tilular 01 Balance: 500.00 WithdrawLimit: 100.00
```

```
Posição da Conta a procurar: 7
```

```
Depósito: 200
```

```
Após Depósito: Number: 09 Holder: Tilular 09 Balance: 700.00 WithdrawLimit: 100.00
```

```
Levantamento: 50
```

```
Após Levantamento: Number: 09 Holder: Tilular 09 Balance: 650.00 WithdrawLimit: 100.00
```

Account
- number : Integer - holder : String - balance : Double - withdrawLimit : Double
+ deposit(amount : Double) : void + withdraw(amount : Double) : void

Exercício de procura

```
class Program {
    0 references
    static void Main(string[] args) {
        Random r = new Random();
        Account[] contas = new Account[10];
        int ix1 = ((int)(r.NextDouble() * 1000)) % contas.Length;

        for (int i = 0; i < contas.Length; i++) {
            contas[i] = new Account();
        }
        // conta a procurar
        Account account = contas[ix1];
        Console.WriteLine("Conta a procurar: " + account);
        Console.WriteLine();
        // baralhar as contas
        Shuffle(contas);
        Console.WriteLine("Contas em carteira:");
        for (int i = 0; i < contas.Length; i++) {
            Console.WriteLine(contas[i]);
        }
        Console.WriteLine();

        int posConta = LinearSearch(contas, account, 0, contas.Length - 1);
        Console.WriteLine("Posição da Conta a procurar: " + posConta);
        if (posConta < 0) {
            return;
        }
        Console.WriteLine();
        Console.WriteLine("Depósito: 200");
        contas[posConta].Deposit(200);
        Console.WriteLine("Após Depósito: " + contas[posConta]);
        Console.WriteLine();
        Console.WriteLine("Levantamento: 200");
        contas[posConta].Withdraw(200);
        Console.WriteLine("Após Levantamento: " + contas[posConta]);
    }
    1 reference
    public static int LinearSearch(Account[] list, Account key, int startIndex, int endIndex) {
        for (int i = startIndex; i <= endIndex; i++) {
            if (list[i].CompareTo(key) == 0) {
                return i;
            }
        }
        return -1;
    }
    1 reference
    public static void Shuffle(Account[] list) {
        Random r = new Random();

        for (int i = 0; i < list.Length; i++) {
            int ix1 = ((int)(r.NextDouble() * 1000)) % list.Length;
            Account tmpAcc = list[ix1];
            int ix2 = ((int)(r.NextDouble() * 1000)) % list.Length;
            list[ix1] = list[ix2];
            list[ix2] = tmpAcc;
        }
    }
}
```

```
class Account : IComparable {
    6 references
    private static int counter = 0;
    2 references
    public int Number { get; set; }
    5 references
    public string Holder { get; set; }
    3 references
    public double Balance { get; set; }
    public double WithdrawLimit { get; set; }

    1 reference
    public Account(int number, string holder, double balance, double withdrawLimit) {
        Number = number;
        Holder = holder;
        Balance = balance;
        WithdrawLimit = withdrawLimit;
    }

    1 reference
    public Account() : this(++counter, String.Format("Titular {0:D2}", counter), 500.00, 100.00) {
    }

    1 reference
    public void Deposit(double d) {
        if (d > Balance) return;
        Balance += d;
    }
    1 reference
    public void Withdraw(double d) {
        if (d > WithdrawLimit) return;
        Balance -= d;
    }

    0 references
    public override string ToString() {
        string s = String.Format("Number: {0:D2} Holder: {1} Balance: {2:F2} WithdrawLimit: {3:F2}", Number, Holder, Balance, WithdrawLimit);
        return s.ToString();
    }

    1 reference
    public int CompareTo(object obj) {
        if (!(obj is Account)) return -1;
        Account a = obj as Account;
        if (Number == a.Number) return 0;
        return Math.Sign(Number - a.Number);
    }
}
```